1. I tried various delta q values and here are the error results for each of the delta q value:

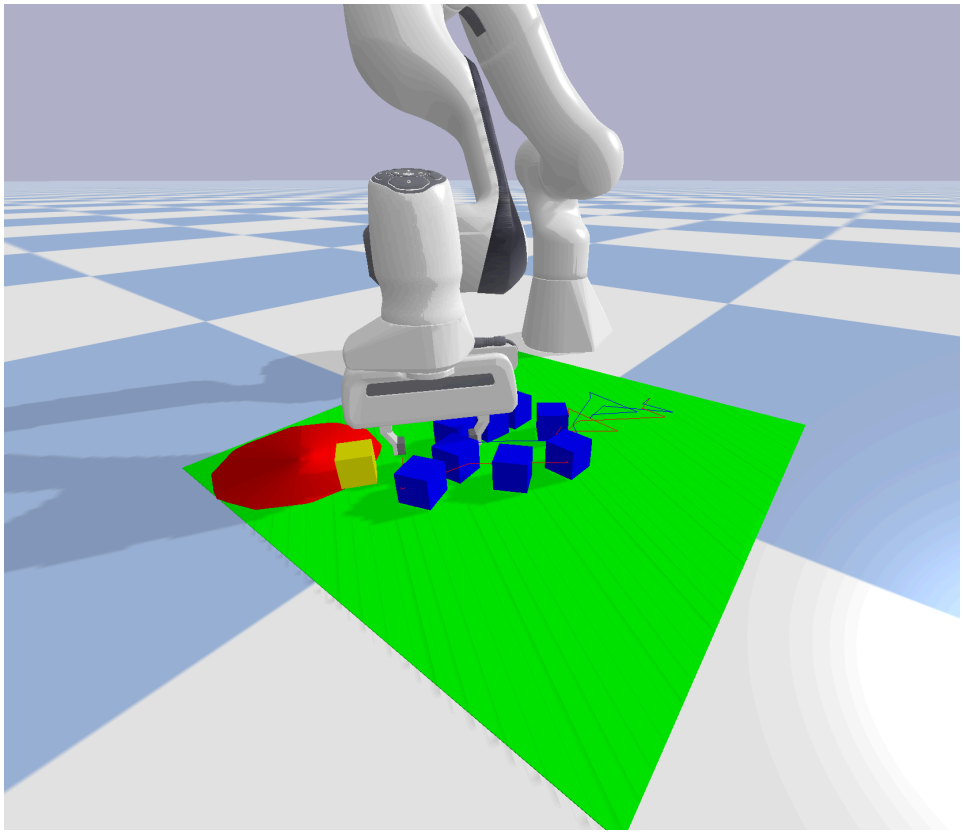
Delta Q	Position Error	Orientation Error
0.0001	0.000352 m	0.008556 rads
0.0005	0.000338 m	0.008281 rads
0.001	0.000322 m	0.008282 rads
0.005	0.000387 m	0.008544 rads
0.01	0.000674 m	0.007759 rads
0.05	Crash	Crash
0.1	Crash	Crash

   With large delta q values such as 0.05 and 0.1, the simulation crashes due to numerical instabilities. The best performance was achieved with delta q = 0.001. At delta q = 0.001 it balances both the position and orientation error. While delta q = 0.01 has lower orientation error, the position error is about 2 times of the other delta q.

2. is_state_valid checks the joint physical limits to prevent the robotic arm from touching the ground and to keep the end effector within the workspace. is_state_high_quality checks to see if the robot is in a singular configuration by calculating the Jacobian. If either checks fail during execution, then the robot stops its motion. is_state_valid checks every 0.2 seconds and is_state_high_quality checks every execution step.

3. The trajectory optimization uses a stochastic, derivative-free local search initialized from a feasible RRT plan. At each iteration, it perturbs one control command with Gaussian noise, re-simulates the full trajectory from the same start state, and rejects candidates that violate validity constraints or fail the task goal. A candidate is accepted only if it reduces a cost that combines end-effector path length and disturbance to non-target objects, so optimization is greedy and monotonic. This produces a practical post-processing step that improves trajectory quality while preserving feasibility in cluttered manipulation scenes.

   In the first image, it shows the original unoptimized path in red and the optimized path in blue. The blue path takes a less jagged path towards the end goal while the red path is more jagged and doesn't always go straight to the end goal. In the execution log, it shows that after the initial path is found, the optimizer gradually decreases the cost of getting to the end goal over time with the final output having the lowest cost.



```
Initial plan found (14 nodes). Starting trajectory optimization...
Optimization start: cost=1.5816  nodes=14
  iter   47: cost improved → 1.5239
  iter   51: cost improved → 1.4943
  iter   52: cost improved → 1.3818
  iter  109: cost improved → 1.3788
  iter  133: cost improved → 1.3573
  iter  187: cost improved → 1.3127
  iter  206: cost improved → 1.2827
  iter  243: cost improved → 1.2691
  iter  321: cost improved → 1.2548
  iter  355: cost improved → 1.1881
  iter  390: cost improved → 1.1778
  iter  459: cost improved → 1.1692
  iter  487: cost improved → 1.1537
Optimization done:  cost=1.1537  nodes=14
Optimization running time: 327.234071 secs
Initial cost: 1.5816 | Optimized cost: 1.1537
```